

03 — Componenti Principali del Cluster OCP

Modulo: 1 — Introduzione e Architettura

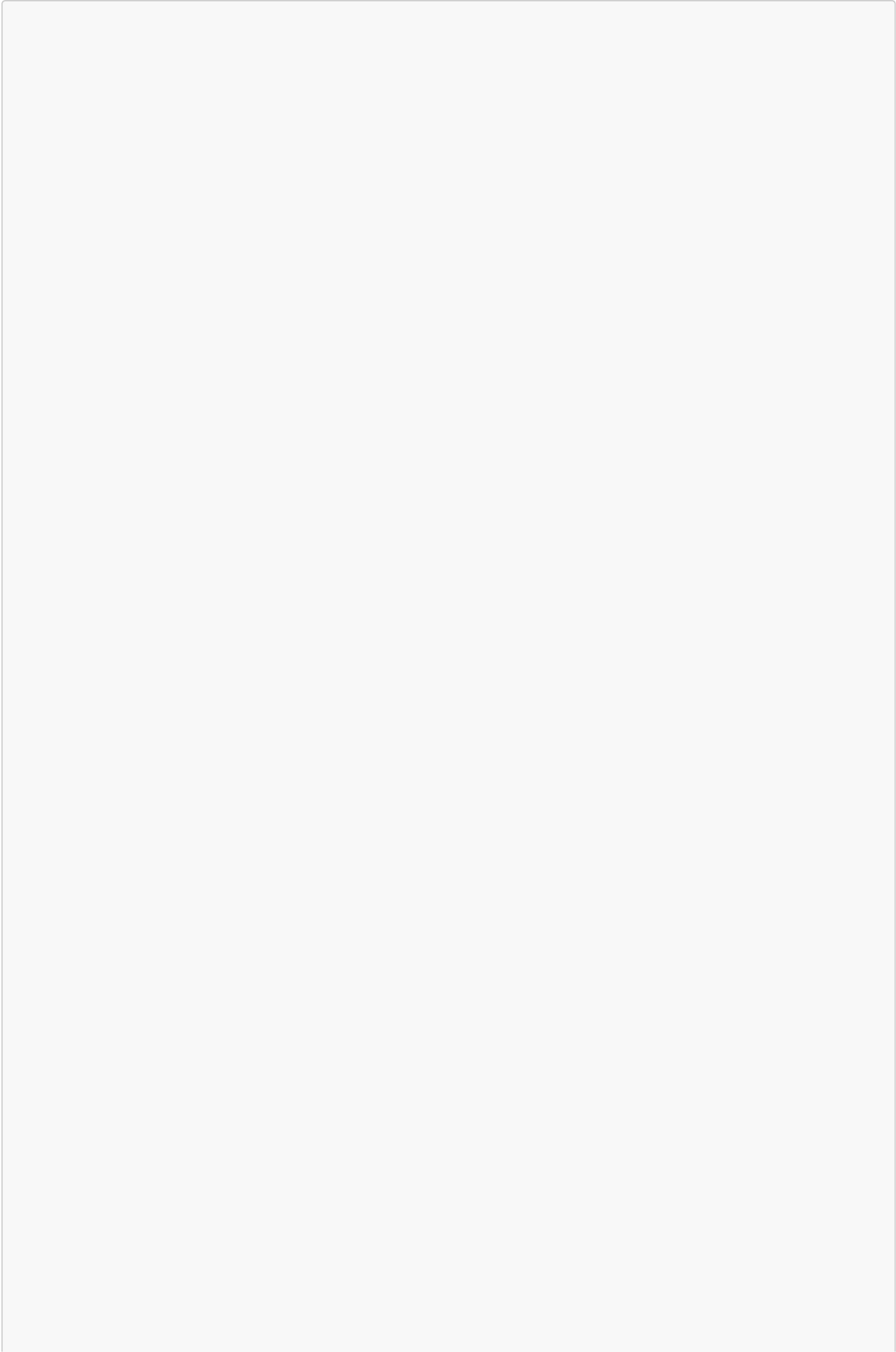
Versione: OpenShift 4.x su Azure ARO

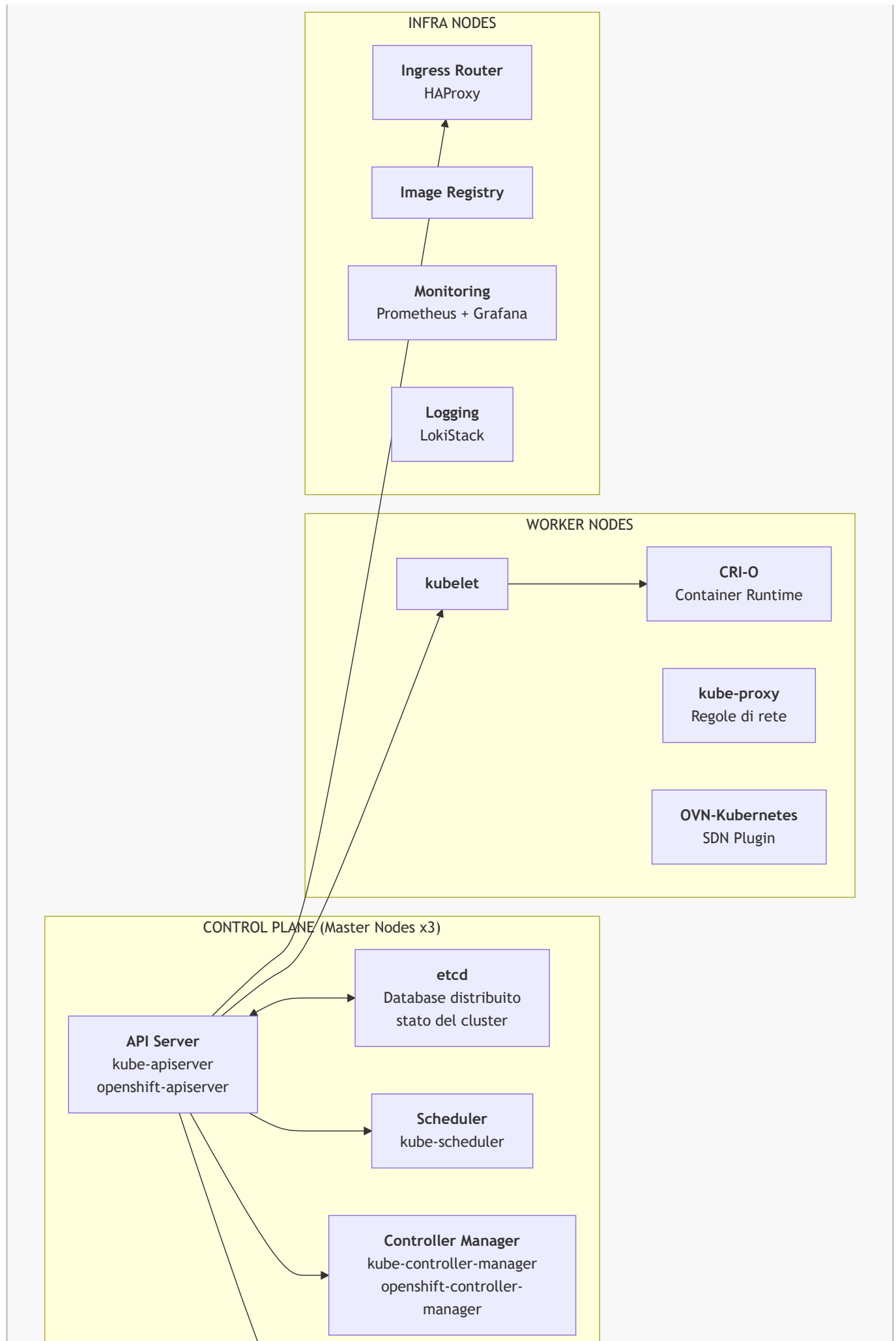
Obiettivi di Apprendimento

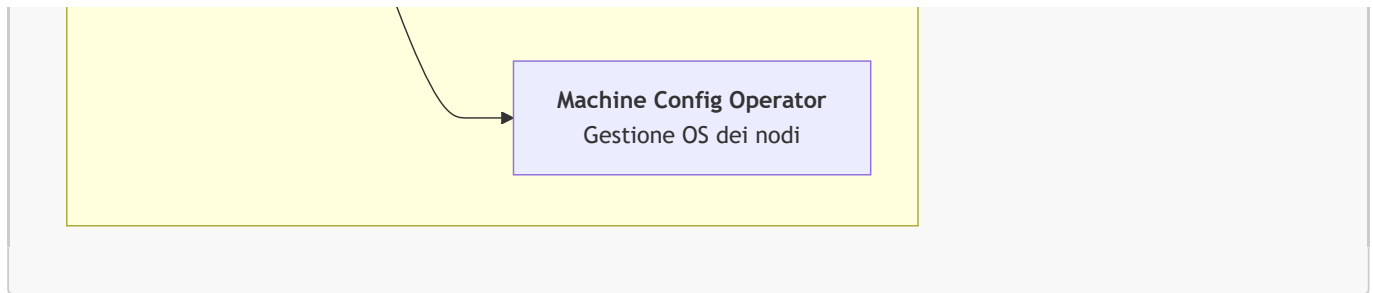
Al termine di questa lezione, il partecipante sarà in grado di:

- Identificare e descrivere il ruolo di ogni componente del Control Plane
 - Comprendere le responsabilità dei Worker Node e degli Infra Node
 - Eseguire comandi diagnostici per verificare lo stato dei componenti
 - Riconoscere le differenze di gestione dei componenti su Azure ARO
-

1. Mappa dei Componenti







2. API Server

Ruolo

L'**API Server** è il **punto di ingresso unico** per tutte le operazioni sul cluster. Ogni componente (kubectl, kubelet, controller) comunica esclusivamente tramite l'API Server.

Caratteristiche

- Espone le API RESTful di Kubernetes e OpenShift
- Gestisce autenticazione, autorizzazione e admission control
- Stateless: può scalare orizzontalmente (in OCP 4.x gira su ogni Master Node)
- Comunica con etcd per leggere/scrivere lo stato

Namespace in OCP

```
# Verificare i Pod dell'API Server
oc get pods -n openshift-kube-apiserver
# Output atteso:
# NAME                                                    READY   STATUS    RESTARTS   AGE
# kube-apiserver-master-0.cluster.example.com            5/5     Running   0          10d
# kube-apiserver-master-1.cluster.example.com            5/5     Running   0          10d
# kube-apiserver-master-2.cluster.example.com            5/5     Running   0          10d

# Verificare i log dell'API Server
oc logs -n openshift-kube-apiserver \
  kube-apiserver-master-0.cluster.example.com --tail=50

# Verificare lo stato dell'operatore che gestisce l'API Server
oc get clusteroperator kube-apiserver
# Output atteso:
# NAME           VERSION   AVAILABLE   PROGRESSING   DEGRADED   SINCE   MESSAGE
# kube-apiserver 4.14.x    True        False         False      10d
```

Admission Controllers

I plugin di Admission Control validano e modificano le richieste prima della persistenza in etcd:

Controller	Funzione
NamespaceLifecycle	Blocca operazioni su namespace in eliminazione

Controller	Funzione
LimitRanger	Applica limiti di risorse default
ServiceAccount	Aggiunge ServiceAccount ai Pod
PodSecurity	Valida Pod Security Standards
SecurityContextConstraint	Applica SCC OpenShift (specifico OCP)
MutatingAdmissionWebhook	Plugin mutanti custom
ValidatingAdmissionWebhook	Plugin validanti custom

3. etcd

Ruolo

etcd è il database distribuito key-value che conserva l'**intero stato del cluster** in modo persistente e affidabile.

Caratteristiche

- Basato su **Raft consensus algorithm** per garantire consistenza
- In OCP 4.x: un'istanza etcd per ogni Master Node (3 istanze = quorum)
- Solo l'API Server scrive direttamente su etcd
- Richiede alta latenza di rete bassa tra i Master Node (< 10ms consigliato)

Quorum e Tolleranza ai Guasti

Master Node attivi	Quorum	Cluster funzionante
3/3	✓	✓
2/3	✓	✓
1/3	✗	✗ (cluster bloccato)

```
# Verificare i Pod etcd
oc get pods -n openshift-etcd
# Output atteso:
# NAME                                READY   STATUS    RESTARTS   AGE
# etcd-master-0.cluster.example.com   4/4     Running   0           10d
# etcd-master-1.cluster.example.com   4/4     Running   0           10d
# etcd-master-2.cluster.example.com   4/4     Running   0           10d

# Verificare la salute del cluster etcd
oc get etcd -o=jsonpath='{range .items[0].status.conditions[*]}{.type}{"\t"}{.status}{"\n"}{end}'

# Verificare l'operatore etcd
oc get clusteroperator etcd
```

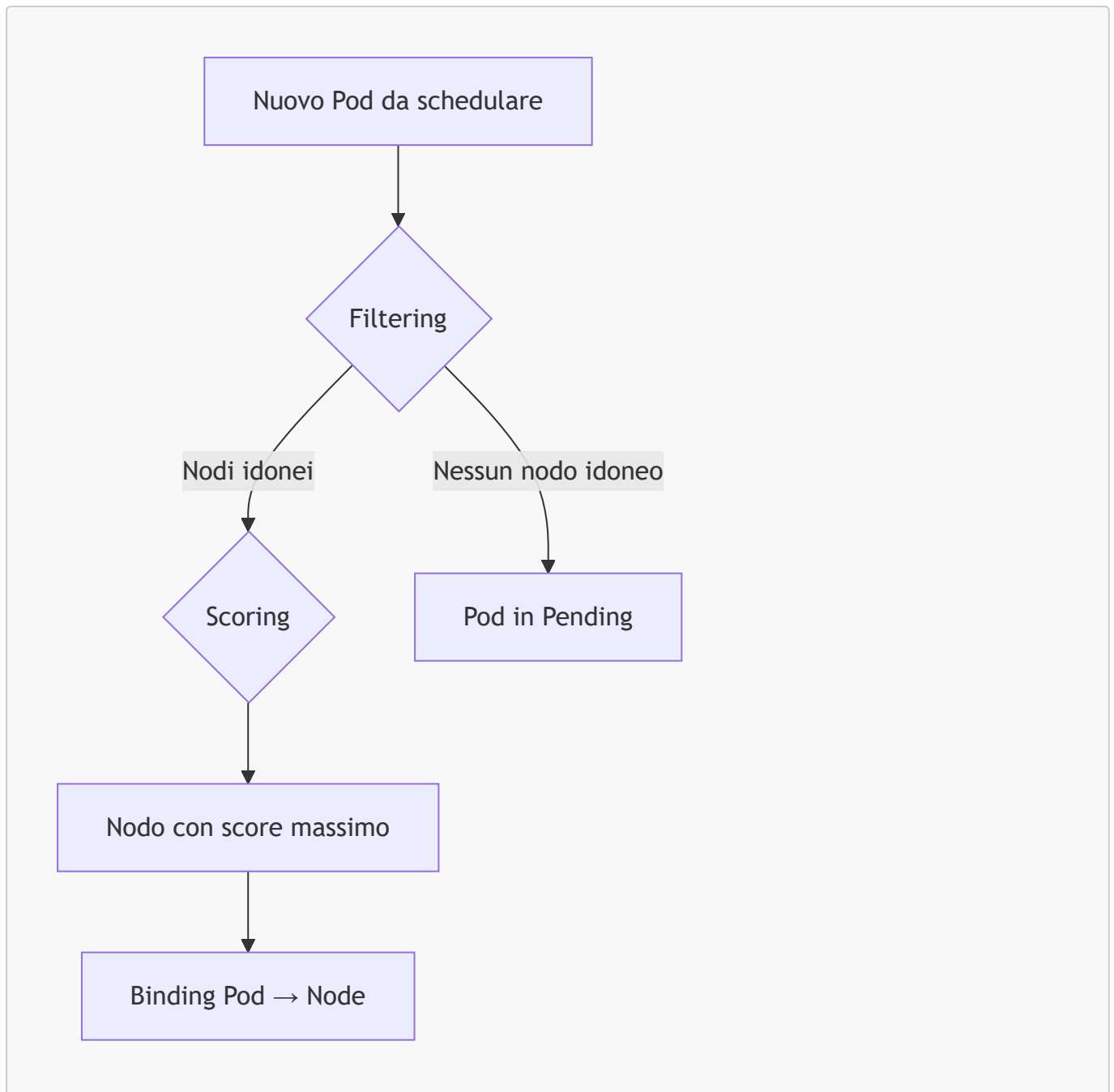
Nota ARO: In Azure ARO il backup di etcd è gestito automaticamente dal team SRE Red Hat/Microsoft. Non è necessario (né possibile) configurare backup manuali di etcd.

4. Scheduler

Ruolo

Il **Scheduler** decide su quale Worker Node eseguire i Pod non ancora assegnati, analizzando le risorse disponibili e le policy configurate.

Processo di Scheduling



Fattori di Filtering (eliminazione nodi non idonei)

- **Risorse:** richieste CPU/RAM del Pod vs disponibili sul nodo
- **NodeSelector / nodeAffinity:** label sui nodi
- **Taints e Tolerations:** nodi "riservati"
- **PodAffinity / PodAntiAffinity:** co-locazione o separazione di Pod
- **Volume:** PV disponibile nella stessa zona del nodo

Esempi Pratici

```
# Visualizzare le risorse disponibili per nodo
oc adm top nodes
# Output atteso:
# NAME                                CPU(cores)   CPU%   MEMORY(bytes)  MEMORY%
# worker-0.cluster.example.com       850m         21%    4Gi            52%
```

```
# worker-1.cluster.example.com 320m 8% 3Gi 38%

# Verificare perché un Pod è in Pending
oc describe pod my-pending-pod | grep -A 10 "Events:"
# Esempio di output:
# Events:
#   Warning FailedScheduling 0/2 nodes available:
#     1 Insufficient cpu, 1 node(s) had taint {node-
# role.kubernetes.io/master: ""}

# Forzare un Pod su un nodo specifico (NodeSelector)
# Nel YAML del Pod:
# spec:
#   nodeSelector:
#     kubernetes.io/hostname: worker-1.cluster.example.com
```

5. Controller Manager

Ruolo

Il **Controller Manager** esegue i **loop di riconciliazione** (reconciliation loops): confronta continuamente lo stato desiderato (in etcd) con lo stato reale del cluster e applica le azioni necessarie per allinearli.

Controller Principali

Controller	Funzione
ReplicaSet Controller	Mantiene il numero di repliche desiderato
Deployment Controller	Gestisce rolling update e rollback
Node Controller	Monitora lo stato dei nodi, applica NotReady
Service Controller	Crea/aggiorna LoadBalancer su cloud provider
Namespace Controller	Pulizia risorse nei namespace eliminati
Job Controller	Gestisce completamento dei Job
CronJob Controller	Crea Job al momento schedato

Controller Specifici OpenShift (openshift-controller-manager)

Controller	Funzione
BuildConfig Controller	Esegue le build S2I/Docker
DeploymentConfig Controller	Gestisce i rollout dei DeploymentConfig
ImageStream Controller	Aggiorna ImageStream quando nuove immagini sono disponibili
Route Controller	Aggiorna la configurazione del router


```
# Verificare i Pod del Controller Manager
oc get pods -n openshift-kube-controller-manager
oc get pods -n openshift-controller-manager

# Verificare l'operatore
oc get clusteroperator kube-controller-manager
```

6. Worker Nodes

6.1 kubelet

L'agente principale su ogni nodo che:

- Riceve le specifiche dei Pod dall'API Server
- Avvia, ferma e monitora i container tramite CRI-O
- Riporta lo stato del nodo e dei Pod all'API Server
- Esegue i **liveness** e **readiness probe**

```
# Verificare lo stato dei nodi
oc get nodes -o wide
# Output atteso:
# NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    OS-
IMAGE
# worker-0.example.com               Ready    worker   10d    v1.27.x    10.0.1.5       RHCOS
4.14
# worker-1.example.com               Ready    worker   10d    v1.27.x    10.0.1.6       RHCOS
4.14

# Accedere ai log del kubelet (tramite debug node)
oc debug node/worker-0.example.com -- chroot /host journalctl -u kubelet --tail=50
```

6.2 CRI-O

CRI-O (Container Runtime Interface for OCI) è il container runtime di OCP 4.x, più leggero di Docker e ottimizzato per Kubernetes:

- Implementa la Container Runtime Interface (CRI)
- Gestisce il ciclo di vita dei container OCI-compliant
- Si integra con CNI (Container Network Interface) per il networking
- Gestisce i layer delle immagini tramite overlay filesystem

```
# Verificare la versione di CRI-O su un nodo
oc debug node/worker-0.example.com -- chroot /host crictl version
# Output atteso:
# Version: 0.1.0
# RuntimeName: cri-o
```

```
# RuntimeVersion: 1.27.x
# RuntimeApiVersion: v1

# Elencare i container in esecuzione su un nodo
oc debug node/worker-0.example.com -- chroot /host crictl ps
```

6.3 OVN-Kubernetes (rete)

Il plugin di rete predefinito in OCP 4.x che fornisce:

- Routing tra Pod su nodi diversi tramite overlay (Geneve/VXLAN)
- Implementazione delle NetworkPolicy
- Load balancing dei Service tramite kube-proxy / OVN load balancer

7. Infra Nodes

Gli **Infra Node** sono Worker Node con il label `node-role.kubernetes.io/infra` dedicati ai servizi infrastrutturali. In ARO esistono per default e vengono gestiti dal servizio.

Componenti tipicamente su Infra Node

Router (Ingress Controller — HAProxy)

```
# Verificare i Pod del Router
oc get pods -n openshift-ingress
# Output atteso:
# NAME                                READY   STATUS    RESTARTS   AGE
# router-default-7d4b9f8d6b-k9xmj     1/1     Running   0           10d
# router-default-7d4b9f8d6b-p2lmn     1/1     Running   0           10d

# Verificare gli IngressController disponibili
oc get ingresscontroller -n openshift-ingress-operator
```

Internal Registry

```
# Verificare il Registry interno
oc get pods -n openshift-image-registry
# Output atteso:
# NAME                                READY   STATUS    AGE
# cluster-image-registry-operator-xxx  1/1     Running   10d
# image-registry-xxx                   1/1     Running   10d

# > Nota ARO: Il Registry interno usa Azure Blob Storage come backend
```

Monitoring Stack

```
# Verificare i componenti del monitoring
oc get pods -n openshift-monitoring
# Output atteso (principali):
# NAME                                READY   STATUS
# alertmanager-main-0                 6/6     Running
# alertmanager-main-1                 6/6     Running
# prometheus-k8s-0                     6/6     Running
# prometheus-k8s-1                     6/6     Running
# grafana-xxx                         2/2     Running
# thanos-querier-xxx                   6/6     Running
```

8. Machine Config Operator (MCO)

Il **MCO** è un componente specifico OCP che gestisce la configurazione del sistema operativo (**RHCOS** — Red Hat CoreOS) su tutti i nodi:

- Applica configurazioni OS tramite **MachineConfig** objects
- Gestisce gli aggiornamenti di RHCOS
- Raggruppa i nodi in **MachineConfigPool** (master, worker, custom)

```
# Verificare i MachineConfigPool
oc get machineconfigpool
# Output atteso:
# NAME      CONFIG                UPDATED   UPDATING   DEGRADED
# master    rendered-master-xxx   True      False      False
# worker    rendered-worker-xxx   True      False      False

# Visualizzare le MachineConfig applicate
oc get machineconfig | head -20
```

9. Note Specifiche per Azure ARO

Componente	Comportamento su ARO
Master Nodes	Gestiti da Red Hat/Microsoft, non accessibili via SSH
etcd backup	Automatico, gestito dal SRE team
Infra Nodes	Presenti per default, gestiti dal servizio
Worker Nodes	Gestibili dall'amministratore (scaling, configurazione)
MCO	Attivo ma con alcune restrizioni sulle MachineConfig
OS dei nodi	RHCOS (immutabile, gestito da MCO)
Storage	Azure Disk (RWO) e Azure Files (RWX) come StorageClass default

```
# Visualizzare i nodi ARO con ruoli
oc get nodes --show-labels | grep node-role

# Scalare i Worker Node su ARO tramite Azure CLI
az aro update \
  --name myAROCluster \
  --resource-group myResourceGroup \
  --worker-count 5
```

10. Riepilogo dei Punti Chiave

- L'**API Server** è il punto di ingresso unico: tutti i componenti comunicano tramite di esso
- **etcd** conserva lo stato del cluster; richiede quorum (minimo 2/3 Master attivi)
- Lo **Scheduler** sceglie il nodo ottimale per ogni Pod basandosi su risorse e policy
- Il **Controller Manager** mantiene lo stato desiderato tramite loop di riconciliazione continui
- **CRI-O** ha sostituito Docker come container runtime in OCP 4.x
- Il **MCO** gestisce RHCOS (sistema operativo dei nodi) in modo dichiarativo
- In **ARO** i Master Node e gli Infra Node sono gestiti dal servizio; i Worker Node sono sotto controllo dell'amministratore

11. Domande di Verifica

1. Se un Master Node si guasta in un cluster a 3 Master, il cluster continua a funzionare? Quale componente determina questa risposta?
2. Qual è la differenza tra il ruolo del **Scheduler** e del **Controller Manager** nella gestione dei Pod?
3. Perché OCP 4.x usa **CRI-O** invece di Docker come container runtime?
4. Cosa succede al sistema operativo di un Worker Node quando viene applicata una **MachineConfig**?
5. In un cluster ARO, un amministratore vuole aumentare il numero di Worker Node. Come procede? Può farlo direttamente su OpenShift?

Riferimenti

- [OCP Control Plane Components](#)
- [Machine Config Operator](#)
- [CRI-O Documentation](#)
- [ARO Node Management - Microsoft](#)
- [etcd Operator - Red Hat](#)